R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

R.S.M. Nagar, PUDUVOYAL-601 206

Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

LAB RECORD

22CS602 – OBJECT ORIENTED SOFTWARE ENGINEERING (LAB INTEGRATED)

Academic Year	**:**	**2025-2026**
Regulations	**:**	**2022**
Batch	**:**	**2023-2027**
Year / Semester	**:**	**III / VI**

Institute Vision and Mission

Vision

To be knowledge hub of providing quality technical education and promoting research for building up of our nation and its contribution for the betterment of humanity

Mission

- To make the best use of state-of-the-art infrastructure to ensure quality technical education
- To develop industrial collaborations to promote innovation and research capabilities
- To inculcate values and ethics to serve humanity

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

R.S.M. Nagar, PUDUVOYAL-601 206

Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision

To be a globally recognized technological Institution fostering excellence in academics, research, and entrepreneurship , producing socially responsible engineers to meet evolving technological changes leading to the sustainable development of the nation and the world.

Mission

- To impart quality technical education through outcome-based learning and modern teaching practices.
- To promote research, innovation, and entrepreneurship through strong industry interaction and incubation initiatives..
- To enhance employability and promote sustainability through continuous skill development and responsible social engagement.

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

R.S.M. Nagar, PUDUVOYAL-601 206

Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Program Educational Objectives (PEOs)

Graduates of Artificial Intelligence and Data Science will

PEO1: Work effectively in inter-disciplinary field with the knowledge of Artificial Intelligence and Data Science to develop appropriate solutions to the real-world problems.

PEO2: Excel in professional career and pursue higher education in the field of Artificial Intelligence and Data Science.

PEO3: Apply their knowledge to the technological revolution through life-long learning.

PEO4: Excel as socially committed engineers or entrepreneurs with high ethical and moral values.

Program Specific Outcomes (PSOs)

Graduates of Artificial Intelligence and Data Science will be able to:

PSO1: Provide solutions to technological problems by using the concepts of Machine Learning and Deep Learning.

PSO2: Analyze and evaluate bigdata to provide effective solutions for complex real-world problems.

PSO3: Create AI-powered innovative solutions for industrial/societal problems.

R.M.K. COLLEGE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

R.S.M. Nagar, PUDUVOYAL-601 206

Approved by AICTE, New Delhi /Affiliated to Anna University, Chennai
Accredited by NBA, New Delhi (All Eligible Courses)/ NAAC with 'A' Grade
An ISO 21001:2018 Certified Institution

22CS602	OBJECT ORIENTED SOFTWARE ENGINEERING (Lab Integrated)	L	T	P	C
		3	0	2	3

OBJECTIVES:

The Course will enable learners to:

- Explain software engineering principles and activities involved in building large software programs
- Describe the process of requirements gathering, analysis and unified modelling
- Illustrate the object-oriented design process
- Analyse various traditional and object-oriented testing methods
- Apply estimation techniques, schedule project activities and compute pricing

UNIT I	PRODUCT AND PROCESS	9+6

The Nature of Software – Defining the Discipline – The Software Process – Process models – Prescriptive Process Models – Product and Process – Agility and Process – What is an Agile Process - Scrum – Other Agile Frameworks – Kanban – DevOps

List of Exercise/Experiments:

1. Identify a software system that needs to be developed.
2. Document the Software Requirements Specification (SRS) for the identified system.

UNIT II	REQUIREMENTS AND UNIFIED MODELING	9 + 6

Requirements Engineering – Establishing the Groundwork: Nonfunctional Requirements – Requirements Gathering – Developing Use Cases – Negotiating and Validating Requirements. Unified Modeling Language – Introduction – Static and Dynamic Models – Modelling – Introduction to the UML – UML Diagrams – UML Class Diagrams – Use-Case Diagram – UML Dynamic Modelling.

List of Exercise/Experiments:

1.Identify use cases and develop the Use Case model.

2.Identify the conceptual classes and develop a Domain Model and also derive a Class Diagram from that.

UNIT III	OBJECT ORIENTED ANALYSIS AND DESIGN	9 + 6

Object oriented Analysis process – Business object Analysis – Use-case driven OOA – Business process modelling – Use case model. Design Concepts – Design Process – Design Concepts – Design Model: Design Principles and Design Elements. Architectural Design – Designing class-based components - Conducting Component Level Design – User Interface Analysis and Design – Pattern-Based Software Design.

List of Exercise/Experiments:

1.Using the identified scenarios, find the interaction between objects and represent them using UML Sequence and Collaboration Diagrams

2.Draw relevant State Chart and Activity Diagrams for the same system

UNIT IV	SOFTWARE TESTING	9 + 6

Component Level: A Strategic Approach to Software Testing – White-Box Testing – Black Box Testing - Object Oriented Testing Integration Level: Integration Testing – AI and Regression Testing – Integration Testing in the OO Context Specialized Testing for Mobility: Web application Testing – Web Testing Strategies – Security Testing – Performance Testing – Real time Testing – Testing AI Systems – Testing Virtual Environments **List of Exercise/Experiments**: 1.Implement the system as per the detailed design 2.Test the software system for all the scenarios identified as per the use-case diagram.		
UNIT V	**SOFTWARE PROJECT MANAGEMENT**	**9 + 6**
Software Metrics and Analytics: Software Measurement – Product Metrics. Creating a Viable Software Plan: The Project Planning Process – Software Scope and Feasibility – Decomposition and Estimation Techniques – Project Scheduling. Risk Management: Reactive Versus Proactive Risk Strategies – Risk Identification – Risk Projection – The RMMM Plan. Software Process Improvement: The SPI Process – The CMMI **List of Exercise/Experiments**: 1.Improve the reusability and maintainability of the software system by applying appropriate design patterns. 2. Implement the modified system and test it for various scenarios.		
	TOTAL PERIODS	**60**
OUTCOMES: After successful completion of the course, the students will be able to		
CO1	Summarize software engineering principles and activities involved in building large software programs	
CO2	Describe the different phases of software development	
CO3	Explain the basics of OOAD and develop software using object oriented design	
CO4	Illustrate the different stages of the design process with a case study	
CO5	Develop mini projects using the application of object oriented analysis and design	
CO6	Apply different testing strategies to develop efficient projects	

COURSE OUTCOMES	
CO1	Summarize software engineering principles and activities involved in building large software programs
CO2	Describe the different phases of software development
CO3	Explain the basics of OOAD and develop software using object oriented design
CO4	Illustrate the different stages of the design process with a case study
CO5	Develop mini projects using the application of object oriented analysis and design
CO6	Apply different testing strategies to develop efficient projects

Course Articulation Matrix

Course Code/ Course Name: 22CS602/ Object Oriented Software Engineering (Lab Integrated)

Course Outcome – Programme Outcome Mapping:

Course Outcomes (COs)	Programme Outcomes (POs)												Programme Specific Outcomes (PSOs)		
	PO1	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PO12**	**PSO1**	**PSO2**	**PSO3**
CO1	3	2	2	1	-	-	1	1	-	-	-	2	1	-	3
CO2	3	3	3	2	-	-	1	1	-	-	-	2	2	-	3
CO3	2	3	3	3	1	-	2	1	1	-	-	3	2	1	2
CO4	2	3	3	3	1	-	2	2	1	-	-	3	3	2	2
CO5	2	3	3	3	2	-	2	3	2	-	2	3	3	3	2
CO6	3	2	3	2	3	-	3	2	2	-	3	3	3	3	3
Average	2.50	2.67	2.83	2.33	1.75	-	1.83	1.67	1.50	-	2.50	2.67	2.33	2.25	2.50

22CS602 – OBJECT ORIENTED SOFTWARE ENGINEERING LABORATORY

LIST OF EXPERIMENTS

S.NO	INDEX	COs Mapped
1.	Study of ARGO UML & UML Diagrams	CO1
2.	Documenting the Software Requirements Specification (SRS) for Quiz System	CO2
3.	Identification of use cases and developing a use case model	CO3
4.	Identification of conceptual classes and develop a domain model to derive a class diagram	CO3
5.	Identifying interaction between objects and developing UML sequence and collaboration diagrams	CO4
6.	Developing UML state chart and activity diagrams	CO4
7.	Implementation of partial layered logical architecture with UML package diagram	CO6
8.	Project development and testing for the quiz system	CO6
9.	Improve the reusability and maintainability of the software system by applying appropriate design patterns	CO5
10.	Implement the modified system and test it for various scenarios	CO5

INTRODUCTION TO OOAD & ArgoUML

What is OOAD?

Object-oriented analysis and design (OOAD) is a software engineering approach that models a system as a group of interacting objects. Each object represents some entity of interest in the system being modeled, and is characterized by its class, its state (data elements), and its behavior. There are a number of different notations for representing these models, such as the Unified Modeling Language (UML).

Object-Oriented analysis

Object-oriented analysis (OOA) looks at the problem domain, with the aim of producing a conceptual model of the information that exists in the area being analyzed. Implementation constraints are dealt during object-oriented design (OOD).

Object-oriented design

Object-oriented design (OOD) transforms the conceptual model produced in object-oriented analysis to take account of the constraints imposed by the chosen architecture and any non-functional – technological or environmental – constraints, such as transaction throughput, response time, run-time platform, development environment, or programming language.

ArgoUML

ArgoUML is an UML diagramming application written in Java and released under the open source Eclipse Public License. By virtue of being a Java application, it is available on any platform supported by Java SE.

ArgoUML is different: i) it makes use of ideas from cognitive psychology, ii) it is based on open standards; iii) it is 100% pure Java; and iv) it is an open source project.

CASE Tools

Computer-aided software engineering (CASE) is the scientific application of a set of tools which is meant to result in high-quality, defect-free, and maintainable software products.

INTRODUCTION TO SELENIUM TESTING TOOL

Introduction

- Selenium is a robust set of tools that supports rapid development of test automation for web-based applications.
- Works anywhere JavaScript is supported
- Hooks for many other languages - Java, Ruby, Python
- Can simulate a user navigating through pages and then assert for specific marks on the pages.
- Selenium IDE is a plug-in to Firefox to record and playback tests (like WinRunner, QTP).

Selenium Components

- Selenium-IDE
- Selenium-RC (Remote Control)
- Selenium-Grid

Steps to start with Selenium!

1) Begin: write and run tests in Firefox.

Selenium IDE is a Firefox add-on that records clicks, typing, and other actions to make a test, which you can play back in the browser.

2) Customize: your language, your browser.

Selenium Remote Control (RC) runs your tests in multiple browsers and platforms. Tweak your tests in your preferred language.

3) Deploy: scale out, speed up

Selenium Grid extends Selenium RC to distribute your tests across multiple servers, saving you time by running tests in parallel.

Ex. No: 01 Date:	STUDY OF ArgoUML & UML DIAGRAMS

Aim: To study about case tools and ArgoUML

ArgoUML

ArgoUML is an open source Unified Modeling Language (UML) modeling tool that includes support for all standard UML 1.4 diagrams. It runs on any Java platform and is available in ten languages.

FEATURES

- Support open standards extensively: UML, XMI, SVG, OCL and others.
- 100% Platform independent thanks to the exclusive use of Java
- Open Source, which allows extending or customizing.
- Cognitive features like
  - reflection-in-action
    - Design Critics
    - Corrective Automations (partially implemented)
    - "To Do" List
    - User model (partially implemented)
  - opportunistic design
    - "To Do" List
    - Checklists
  - Comprehension and Problem Solving
    - Explorer Perspectives
    - Multiple, Overlapping Views
    - Alternative Design Representations: Graphs, Text, or Table

Supported Diagrams by ArgoUML

- Use Case Diagrams
- Class Diagrams
- Behavior Diagrams
  - State chart Diagrams
  - Activity Diagrams
  - Interaction Diagrams
    - » Sequence Diagrams
    - » Collaboration Diagrams
- Implementation Diagrams
  - Component Diagrams
  - Deployment Diagrams

Use Case Diagrams

- Present a high-level view of system usage
- These diagrams show the functionality of a system or a class and how the system interacts with the outside world.
- During analysis to capture the system requirements and to understand how the system should work.

During the design phase, use-case diagrams specify the behavior of the system as implemented.

Class Diagram

- Helps you visualize the structural or static view of a system.
- Class diagrams show the relationships among class.
- Foundation for component and deployment diagrams.

Sequence Diagram

- Illustrates object interacts arranged in a time sequence
- This shows step-by-step what has to happen to accomplish something in the use case and emphasize the sequence of events.

Collaboration Diagram

- Provides a view of the interactions or structured relationships between objects in current model.
- Emphasizes relation between objects.
- Used as the primary vehicle to describe interactions that express decision about system behavior.

State Chart Diagram

- To model the dynamic behaviors of individual classes or objects
- Used to model the discrete stages of an objects lifetime.

Activity Diagram

- Model the workflow of a business process and the sequence of activities in a process.
- Similar to a flowchart, l a workflow from activity to activity or from activity to state.
- It is help to understand the overall process.

Component Diagram

- A physical view of the current model and Show the organization and dependencies of software components, including source code, binary code, and executable components

Deployment Diagram

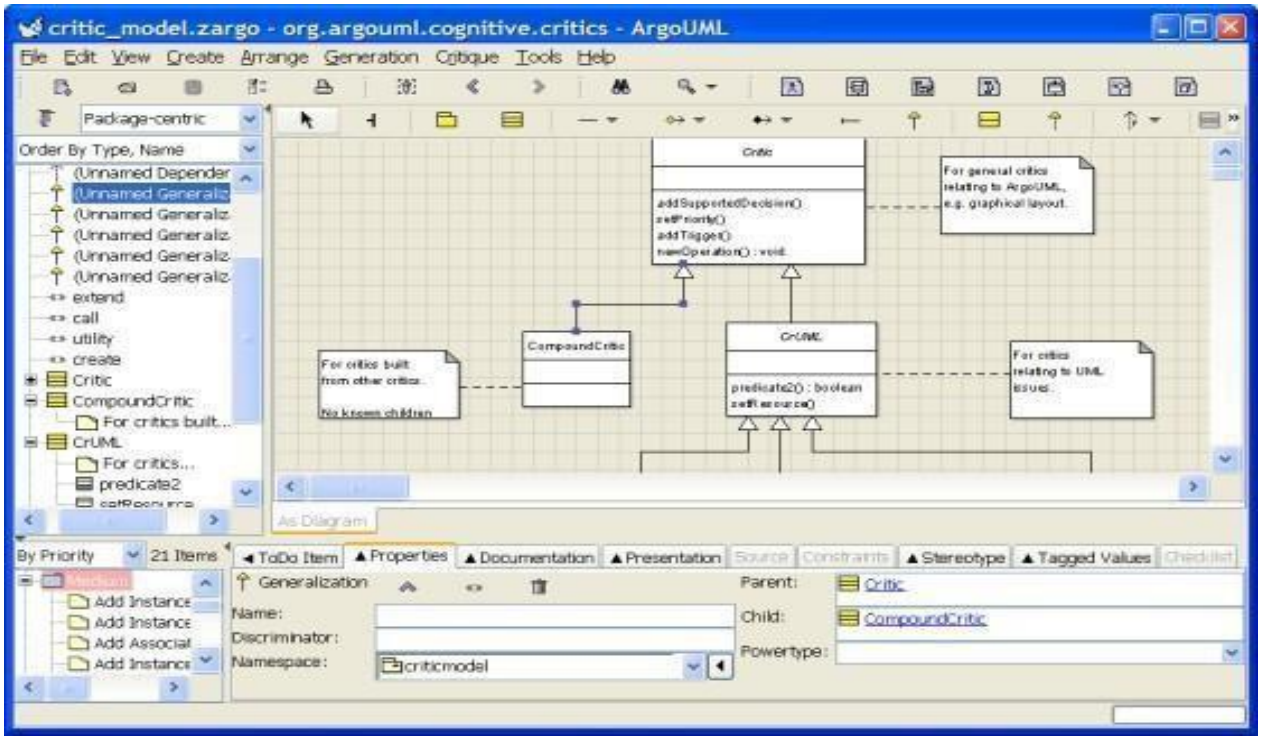
- Each model contains a single deployment diagram that shows the mapping of processes to hardware.

THE ArgoUML USER INTERFACE

Overview of the Window

The title bar of the window shows the following 4 parts of information, separated from each other by a dash.

- The current filename.
- The name of the currently active diagram.
- The name "ArgoUML".
- An asterisk (*). This item is only present if the current project file is "dirty", i.e. it is altered, but not yet saved. In other words, if the asterisk is absent, then the current file has not been altered.



The window comprises four sub-windows or panes namely,

- The Explorer pane,
- The Editing pane
- The Details Pane and
- The To-Do Pane.

THE EDITING PANE

The Toolbar

The toolbar at the top of the editing pane provides the main functions of the pane.

The Toolbar consists of the following menus:

- File operations
- Edit operations
- View operations
- Create operations

File operations

- New
- Open Project...
- Save Project
- Project Properties
- Print

Edit operations

- Remove From Diagram
gram ".
- Delete From Model
- Configure Perspectives
- Settings

View operations

- Find...
- Zoom

Create operations

- New Use Case Diagram
- New Class Diagram
- New Sequence Diagram
- New Collaboration Diagram
- New Statechart Diagram
- New Activity Diagram
- New Deployment Diagram

The tools fall into four categories.

- ***Layout tools.*** Provide assistance in laying out model elements on the diagram.
- ***Annotation tools.*** Used to annotate model elements on the diagram.
- ***Drawing tools.*** Used to add general graphic objects to diagrams.
- ***Diagram specific tools.*** Used to add UML model elements specific to a particular diagram type to the diagram.

- ***<u>Layout Tools</u>***

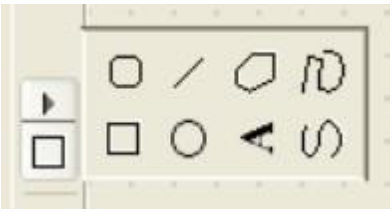
↑ **Select** - This tool provides for general selection of model elements on the diagram.

┤**Broom Tool** – This tool is used to sweep all model elements along.

- ***Annotation Tools***

Annotation tool - It is used to add a comment to a selected UML model element.

- ***Drawing Tools***



- Rectangle. Provides a rectangle.
- Rounded Rectangle. Provides a rectangle with rounded corners. There is no control over the degree of rounding.
- Circle. Provides a circle.
- Line. Provides a line.
- Text. Provides a text box. The text is entered by selecting the box and typing. Text is centered horizontally and after typing, the box will shrink to the size of the text. However it can be re-sized by dragging on the corners.
- Polygon. Provides a polygon. The points of the polygon are selected by button 1 click and the polygon closed with button 1 double click (which will link the final point to the first point).
- Spline. Provide an open spline. The control points of the spline are selected with button 1 and the last point selected with button 1 double click.
- Ink. Provide a polyline. The points are provided by button 1 motion.

USE CASE DIAGRAM SPECIFIC TOOLS

Actor - Add an actor to the diagram.

Use Case – Add a use case to the diagram.

Association - Add an association between two model elements selected using button 1 motion (from the first model element to the second).

The association tool selector

- `Dependency`. Add a dependency between two model elements selected using button 1 motion (from the dependent model element).
- `Generalization`. Add a generalization between two model elements selected using button 1 motion (from the child to the parent).
- `Extend`. Add an extend relationship between two model elements selected using button 1 motion (from the extended to the extending use case).
- `Include`. Add an include relationship between two model elements selected using button 1 motion (from the including to the included use case).
- `Add Extension Point`. Add an extension point to a selected use case.

CLASS DIAGRAM SPECIFIC TOOLS

Class diagrams are used for only one of the UML static structure diagrams, the class diagram itself. Object diagrams are represented on the ArgoUML deployment diagram.

ArgoUML uses the class diagram to show model structure through the use of packages.

- `Package`. Add a package to the diagram.
- `Class`. Add a class to the diagram. For convenience, when the mouse is over a selected class it displays two handles to left and right which may be clicked or dragged to form association relationships (or composition in case SHIFT has been pressed) and two handles top and bottom which may be dragged or clicked to form generalization and specialization relationships respectively.
- `Association`. Add an association between two model elements selected using button 1 motion (from the first model element to the second). There are 2 types of association offered here, `bidirectional` or `unidirectional`.
- `Aggregation`. Add an aggregation between two model elements selected using button 1 motion (from the first model element to the second). There are 2 types of aggregation offered here, `bidirectional` or `unidirectional`.
- `Composition`. Add an composition between two model elements selected using button 1 motion (from the first model element to the second). There are 2 types of composition offered here, `bidirectional` or `unidirectional`.

- `Association-end`. Add another end to an already existing association using button 1 (from the association middle to a class, or vice versa). This is the way to create so-calld N-ary associations.
- `Generalization`. Add a generalization between two model elements selected using button 1 (from the child to the parent).
- `Interface`. Add an interface to the diagram. For convenience, when the mouse is over a selected interface it displays a handle at the bottom which may be dragged to form a realization relationship (the target being the realizing class).
- `Realization`. Add a realization between a class and an interface selected using button 1 motion (from the realizing class to the realized interface).
- `Dependency`. Add a dependency between two model elements selected using button 1 motion (from the dependent model element). There are also 2 special types of dependency offered here, `Permission` (P) and `Usage` (U). A `Permission` is created by default with stereotype `Import`, and is used to import elements from one package into another.
- `Attribute`. Add a new attribute to the currently selected class. The attribute is given the default name `newAttr` of type `int` and may be edited by button 1 double click and using the keyboard, or by selecting with button 1 click (after the class has been selected) and using the property tab.

SEQUENCE DIAGRAM SPECIFIC TOOLS

- `ClassifierRole`. Add a classifierrole to the diagram.
- `Message with Call Action`. Add a call message between two classifierroles selected using button 1 motion (from the originating classifierrole to the receiving classifierrole).
- `Message with Return Action`. Add a return message between two classifierroles selected using button 1 motion (from the originating classifierrole to the receiving classifierrole).
- `Message with Create Action`. Add a create message between two classifierroles selected using button 1 motion (from the originating classifierrole to the receiving classifierrole).
- `Message with Destroy Action`. Add a destroy message between two classifierroles selected using button 1 motion (from the originating classifierrole to the receiving classifierrole).
-

Add Vertical Space to Diagram. Add vertical space to a diagram by moving all messages below this down. Click the mouse at the point where you want the space to be added and drag down the screen vertically the distance which matches the height of the space you'd like to have added.

- Remove Vertical Space in Diagram. Remove vertical space from diagram and move all elements below up vertically. Click and drag the mouse vertically over the space that you want deleted.

COLLABORATION DIAGRAM SPECIFIC TOOLS

- Classifier Role. Add a classifier role to the diagram.
- Association Role. Add an association role between two classifier roles selected using button 1 motion (from the originating classifier role to the receiving classifier role). There are 6 types of association roles offered here, see Figure 12.4, " The association tool selector. ": association, aggregation and composition, and all these three can be bidirectional or unidirectional.
- Generalization. Add a generalization between two model elements selected using button 1 (from the child to the parent).
- Dependency. Add a dependency between two model elements selected using button 1 motion (from the dependent model element).
- Add Message. Add a message to the selected association role.

STATE CHART DIAGRAM SPECIFIC TOOLS

- Simple State. Add a simple state to the diagram.
- Composite State. Add a composite state to the diagram.
- Transition. Add a transition between two states selected using button 1 motion
- Synch State. Add a synchstate to the diagram.
- Submachine State. Add a submachinestate to the diagram.
- Stub State. Add a stubstate to the diagram.
- Initial. Add an initial pseudostate to the diagram.

- Shallow History. Add a shallow history pseudostate to the diagram.
- Deep History. Add a deep history pseudostate to the diagram.
- Call Event. Add a Call Event as trigger to a transition. There are 4 types of events offered here: Call Event, Change Event, Signal Event and Time Event.
- Guard. Add a guard to a transition.
- Call Action. Add a call action (i.e. the effect) to a transition.

ACTIVITY DIAGRAM SPECIFIC TOOLS

- Action State. Add an action state to the diagram.
- Transition. Add a transition between two action states selected using button 1 motion (from the originating action state to the receiving action state).
- Initial. Add an initial pseudostate to the diagram.
- Final State. Add a final state to the diagram.
- Junction. Add a junction (decision) pseudostate to the diagram.
- Fork. Add a fork pseudostate to the diagram.
- Join. Add a join pseudostate to the diagram.
- CallState. Add a callstate to the diagram. A call state is an action state that calls a single operation. Hence, the name of the operation being called is put in the symbol, along with the name of the classifier that hosts the operation in parentheses under it.
- ObjectFlowState. Add a objectflowstate to the diagram. An objectflowstate is an object that is input to or output from an action.

DEPLOYMENT DIAGRAM SPECIFIC TOOLS

- Node. Add a node to the diagram. For convenience, when the mouse is over a selected node it displays four handles to left, right, top and bottom which may be dragged to form association relationships.
- Node Instance. Add a node instance to the diagram. For convenience, when the mouse is over a selected node instance it displays four handles to left, right, top and bottom which may be dragged to form link relationships.

Component. Add a component to the diagram. For convenience, when the mouse is over a selected component it displays four handles to left, right, top and bottom which may be dragged to form dependency relationships.

- Component Instance. Add a component instance to the diagram. For convenience, when the mouse is over a selected component instance it displays four handles to left, right, top and bottom which may be dragged to form dependency relationships.
- Generalization. Add a generalization between two model elements selected using button 1 (from the child to the parent).
- Realization. Add a realization between a class and an interface selected using button 1 motion (from the realizing class to the realized interface).
- Dependency. Add a dependency between two model elements selected using button 1 motion (from the dependent model element).
- Association. Add an association between two model elements

RESULT: Thus, the ArgoUML software has been successfully installed and the UML tools to implement the various UML diagrams are learnt.

ArgoUML Installation and understanding (3 marks)	
UML Implementation (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

Ex. No: 02 Date:	**Documenting the Software Requirements Specification (SRS) for Quiz System**

Aim: To develop a software requirement specification for QUIZ system

PROBLEM STATEMENT

Online Quiz system is to be designed for the students of XYZ college of Technology. The questions should be of objective type with multiple options. The system should be able to handle errors such as 2 options for one answer. After answering the questions in the preliminary round, the system verifies whether the student has answered the minimum number of questions in each level. If not, a message is displayed to make the student answer the required number of questions within an allotted time. If the student qualifies the prelims he/she can proceed to the finals in the same way. At the end of the quiz the users' score along with information whether he has been selected or not has to be displayed. The quiz system has two levels of questions.

LEVELS	TOTAL NO. OF QUESTIONS	MINIMUM NO. OF QUESTIONS TO BE ATTENDED	MARKS FOR EACH QUESTION	DURATION
LEVEL1-PRELIMINARY	30	25	1	15 Min
LEVEL2-FINALS	15	10	2	10 Min

RESULT: Thus, the problem statement and the document for software requirement specification for the Quiz System has been successfully prepared.

Understanding of Problem Statement (3 marks)	
Preparation of SRS Document (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

Ex. No: 03 Date:	IDENTIFICATION OF USE CASES AND DEVELOPING A USE CASE MODEL

Aim: To identify Use Cases and develop the Use Case model for QUIZ system

Description:

Guideline: To Find Use Cases

Use cases are defined to satisfy the goals of the primary actors. Hence, the basic procedure is:

Step 1: Choose the system boundary. Is it just a software application, the hardware and application as a unit, that plus a person using it, or an entire organization?

Step 2: Identify the primary actors those that have goals fulfilled through using services of the system. The following questions help identify others that may be missed:

Who starts and stops the system?	Who does system administration?
Who does user and security management?	Is "time" an actor because the system does something in response to a time event?
Is there a monitoring process that restarts the system if it fails?	Who evaluates system activity or performance?
How are software updates handled? Push or pull update?	Who evaluates logs? Are they remotely retrieved?
In addition to human primary actors, are there any	Who gets notified when there are
Who starts and stops the system?	Who does system administration?
External Software or Robotic Systems that call upon services of the system?	Error or failures?

Step 3: Identify the goals for each primary actor.

For example, use this table to prepare actor goal list

Actor	Goal
	Play level 1
Participant	**Play level 2**
	Submit answers
...	**...**

Step 4: Define Use Cases

In general, define one use case for each user goal. Name the use case similar to the user goal. Start the name of use cases with a verb.

A common exception to one use case per goal is to collapse CRUD (create, retrieve, update, delete) separate goals into one CRUD use case, idiomatically called Manage <X>. For example, the goals "edit user," "deleteuser," and so forth are all satisfied by the Manage Users use case.

UML Notation:

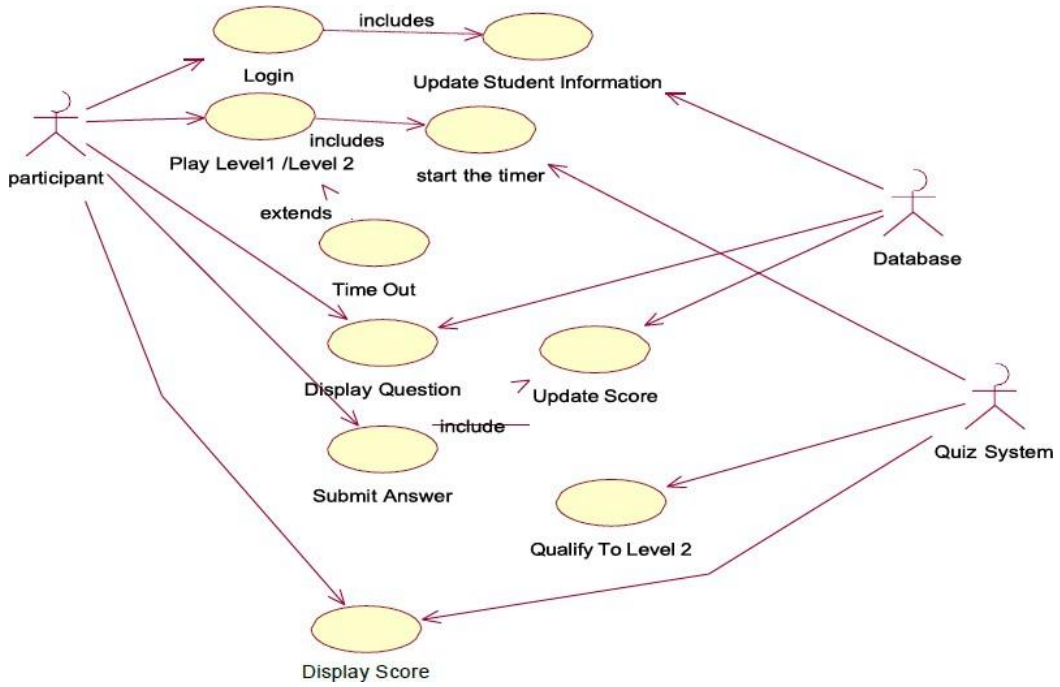
Actor : An Actor, as mentioned, is a user of the system, and is depicted using a stick figure. The role of the user is written beneath the icon. Actors are not limited to humans. If a system communicates with another application, and expects input or delivers output, then that application can also be consideredan actor.

Use Case: A Use Case is functionality provided by the system, typically described as verb+object (e.g. Register Car, Delete User). Use Cases are depictedwith an ellipse. The name of the use case is written within the ellipse.

Association: Associations are used to link Actors with Use Cases, and indicate that an Actor participates in the Use Case in some form. Associations are depicted by a line connecting the Actor and the Use Case.

Actor **Association** **Use case**

Use-case Diagram: Quiz System



3.1 Use cases

The system will consist of Login screen to authenticate the participants whose information is updated in the database. On starting the quiz , timer is started to maintain the timings. In Level 1 /Level 2 Questions with four options are displayed sequentially .User select the answer and move to the next question . Finally he/she selects the Submit answers which updates the marks and displays the score to the participants. If he is playing Level 1 and qualified for level 2 , then next level questions are displayed otherwise Not Qualified Message is displayed. **Functional Requirements**

3.1.1. Use Case: Login

Brief Description

The use case describes how a Participant logs into the Quiz System

Use Case Section	**Comment**
Use Case Name	Login
Scope	Quiz System
Level	"user-goal"
Use Case Section	**Comment**
Primary Actor	Participant
Stakeholders and Interest list	- Participant: logs into the Quiz System . . .
Preconditions	None

Success Guarantee/ Post condition	If the use case was successful, the actor is now logged into the system. If not, the system State is unchanged.
Main Success Scenario	1. The System requests the actor to enter his/her name and password 2. The actor enters his/her name and password 3. The System validates the entered name and password and logs the actor into the System
Extensions	3a.If the pwd is wrong, user is allowed for 3 attempts
Special Requirements	Timer
Technology and Data Variations List	-
Frequency of Occurrence	Could Be nearly Continuous
Miscellaneous	If the connection is terminated before the form is submitted, the fields are cleared and the Server is returned to the wait state. The Administrator can make the system not to get updated by others.

3.1.2. Use Case: Play Level 1 /Level 2 (provide the content same as above

3.1.3. Use Case: Display Questions (provide the content same as above)

3.1.4. Use Case: Submit Answers (provide the content same as above)

3.1.5. Use Case: Qualify to Level 2 (provide the content same as above)

3.1.6. Use Case: Display Score (provide the content same as above)

3.2. Non-functional requirements

Non-functional requirements are often called qualities of a system. Other terms for non-functional requirements are "constraints", "quality attributes", "quality goals", "quality of service requirements" and "non-behavioral requirements".

Non-functional requirements, can be divided into two main categories:

1) Execution qualities, such as security and usability, which are observable at run time.

2) Evolution qualities, such as testability, maintainability, extensibility and scalability, which are embodied in the static structure of the software system.

3.2.1. Functionality : Multiple users must be able to perform their work concurrently. If the participant has completed 30 Minutes allotted for him/her, he or she should be notified with the message "your time slot is over".

3.2.2. Usability: The desktop user-interface shall be Windows 95/98/2000/xp compliant.

3.2.3. Reliability: The System should function properly for allotted time slot and produces score card with no more than 10% down time.

3.2.4. Performance

1. The System shall support up to 100 simultaneous users against the central database at any given time and up to 100 simultaneous users against the local servers at any one time.
2. The System must be able to complete 80% of all transactions within 2 minutes.

3.2. 5. Supportability: None.

3.2. 6. Security

1. The System should secure so that only registered participants can take part in Quiz.
2. Once the participant had submitted a answer, he/she can't change the answer later.

3.2.7. Design Constraints: The system shall provide a window-based desktop interface.

RESULT: Thus, the use case model for the quiz system has been successfully identified and the use case model has been developed for the same using the ArgoUML.

Identification of Use Cases (3 marks)	
Developing a Use Case Model (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

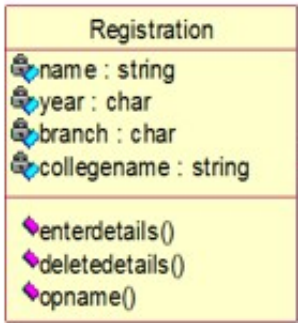
Ex. No: 04 Date:	IDENTIFICATION OF CONCEPTUAL CLASSES AND DEVELOP A DOMAIN MODEL TO DERIVE A CLASS DIAGRAM

Aim: To Identify the Conceptual Classes and to develop a domain model with UML class diagram for QUIZ system

Description

A class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, and the relationships between the classes.

The class diagram is the main building block in object oriented modeling. They are being used both for general conceptual modeling of the systematic of the application, and for detailed modeling translating the models into programming code. The classes in a class diagram represent both the main objects and or interactions in the application and the objects to be programmed. In the class diagram these classes are represented with boxes which contain three parts.

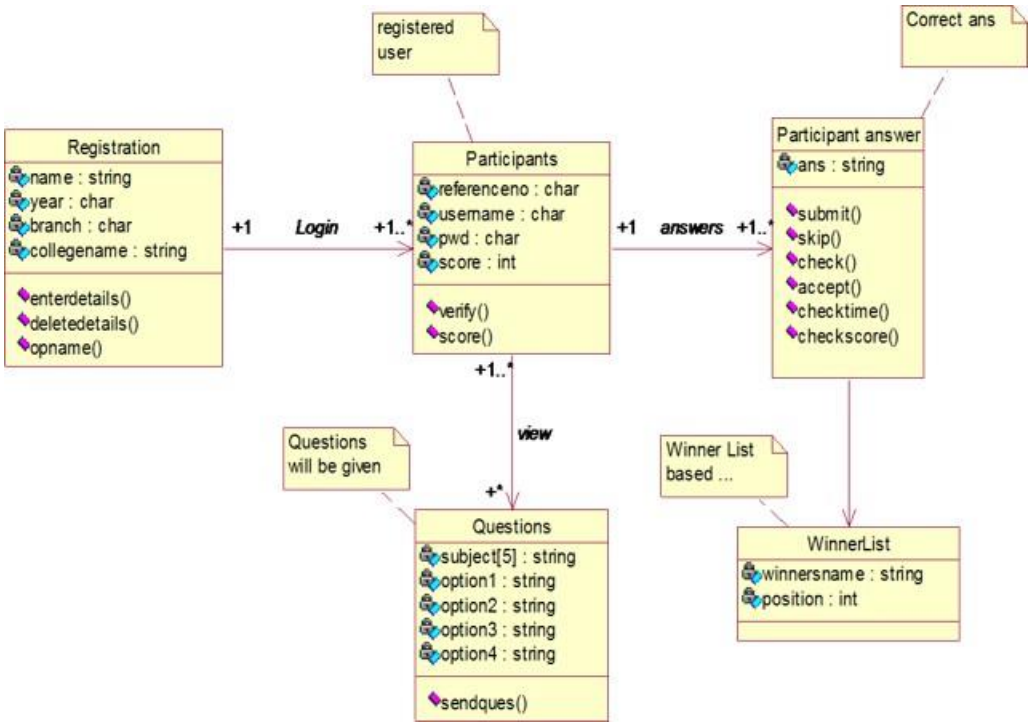


A class with three sections.

- The upper part holds the name of the class
- The middle part contains the attributes of the class
- The bottom part gives the methods or operations the class can take or undertake

In the system design of a system, a number of classes are identified and grouped together in a class diagram which helps to determine the statically relations between those objects. With detailed modeling, the classes of the conceptual design are often split in a number of subclasses.

Class Diagram



RESULT: Thus, the conceptual classes are identified, a domain model has been developed and a class diagram has been derived successfully for the Quiz System using the ArgoUML.

Identification of Conceptual classes and developing a domain model (3 marks)	
Deriving a Class Diagram (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

Ex. No: 05 Date:	IDENTIFYING INTERACTION BETWEEN OBJECTS AND DEVELOPING UML SEQUENCE AND COLLABORATION DIAGRAMS

Aim: To Identify the interaction between objects and to develop using UML Sequence and Collaboration diagrams for QUIZ system

Description

Sequence diagrams document the interactions between classes to achieve a result, such as a use case. Because UML is designed for object- oriented programming, these communications between classes are known as messages. The Sequence diagram lists objects horizontally, and time vertically, and models these messages over time.

Notation: In a Sequence diagram, classes and actors are listed as columns, with vertical lifelines indicating the lifetime of the object over time.

Object: Objects are instances of classes, and are arranged horizontally. The pictorial representation for an Object is a class (a rectangle) with the name prefixed by the object name (optional) and a semi-colon. **: Object1**

Actor: Actors can also communicate with objects, so they too can be listed as a column. An Actor is modeled using the ubiquitous symbol, the stick figure.

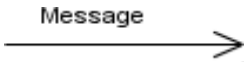
Actor1

Lifeline: The Lifeline identifies the existence of the object over time. The notation for a Lifeline is a vertical dotted line extending from an object.

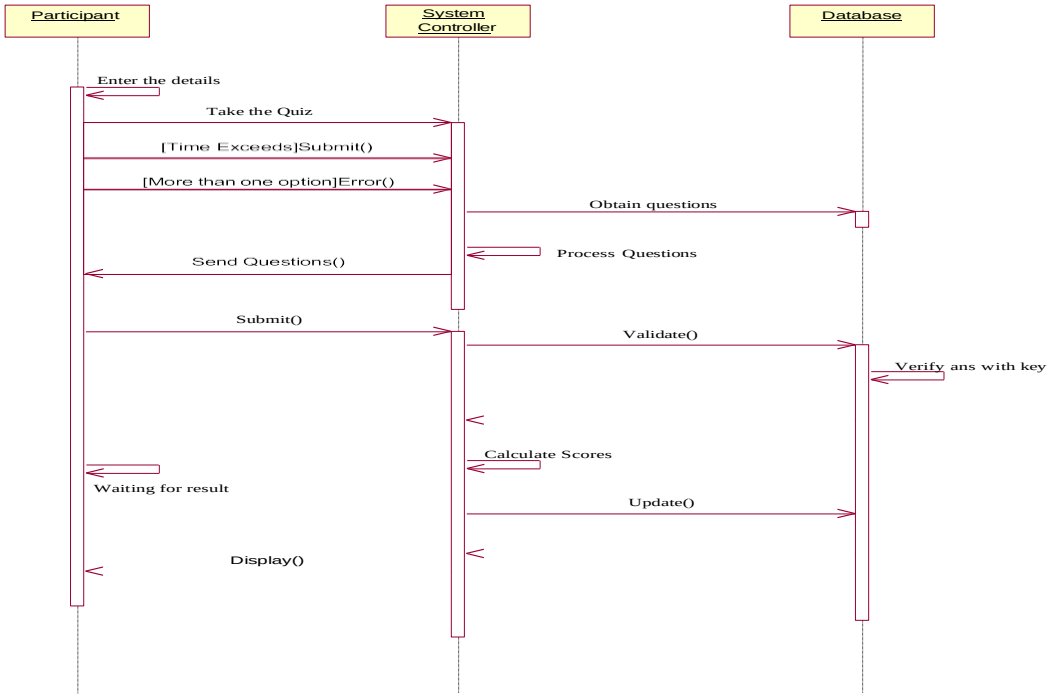
Activation: Activations, modeled as rectangular boxes on the lifeline, indicate when the object is performing an action.

Message

Messages, modeled as horizontal arrows between Activations, indicate the communications between objects.



Sequence diagram



RESULT: Thus, the scenarios are identified with interaction of objects and the UML Sequence and Collaboration Diagrams are represented successfully for the Quiz System using the ArgoUML.

Identification of scenarios and object interaction (3 marks)	
Representation of UML Seuqence and Collaboration Diagrams (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

Ex. No: 06 Date:	DEVELOPING UML STATE CHART AND ACTIVITY DIAGRAMS

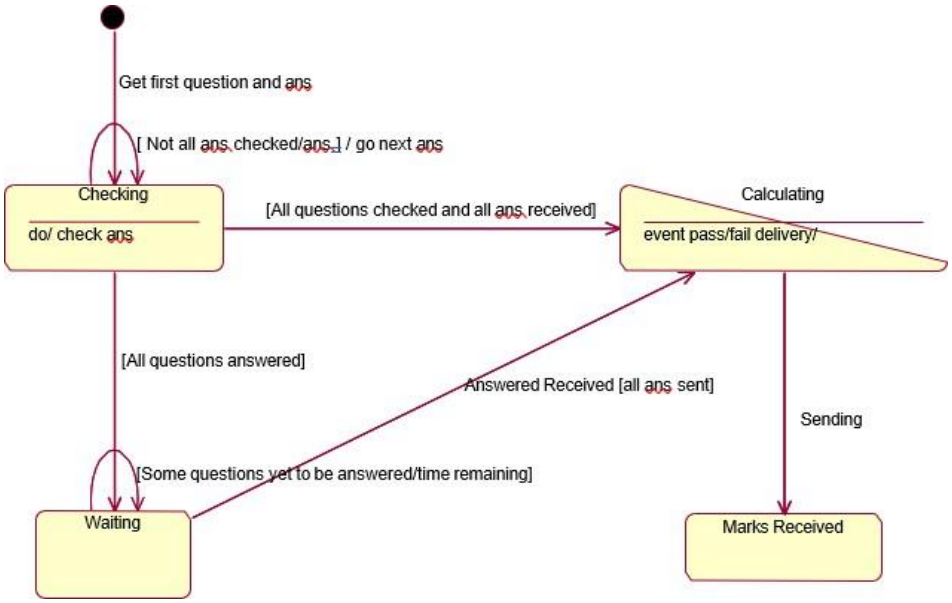
Aim: To develop using UML State Chart diagram and Activity diagrams for QUIZ system

State Chart Diagram Description

UML preserves the general form of the traditional state diagrams. The UML state diagrams are directed graphs in which nodes denote states and connectors denote state transitions. For example, Figure 1 shows a UML state diagram corresponding to the computer keyboard state machine. In UML, states are represented as rounded rectangles labeled with state names. The transitions, represented as arrows, are labeled with the triggering events followed optionally by the list of executed actions. The initial transition originates from the solid circle and specifies the default state when the system first begins. Every state diagram should have such a transition, which should not be labeled, since it is not triggered by an event. The initial transition can have associated actions.
The main purposes of using State chart diagrams:

- To model dynamic aspect of a system.
- To model life time of a reactive system.
- To describe different states of an object during its life time.
- Define a state machine to model states of an object Steps to prepare state chart diagram
- Identify important objects to be analyzed.
- Identify the states.
- Identify the events.

State Chart Diagram



Activity Diagram Description

Activity diagrams are used to document workflows in a system, from the business level down to the operational level. Activity diagram is a variation of the state diagram where the "states" represent operations, and the transitions represent the activities that happen when the operation is complete. The general purpose of Activity diagrams is to focus on flows driven by internal processing vs. external events.

Activity States

Activity states mark an action by an object. The notations for these states are rounded rectangles, the same notation as found in State chart diagrams.

Transition: When an Activity State is completed, processing moves to another Activity State. Transitions are used to mark this movement. Transitions are modeled using arrows.

Swim lane: Swim lanes divide activities according to objects by arranging objects in column format and placing activities by that object within that column. Objects are listed at the top of the column, and vertical bars separate the columns to form the swim lanes.

Initial State: The Initial State marks the entry point and the initial Activity State. The notation for the Initial State is the same as in State chart diagrams,

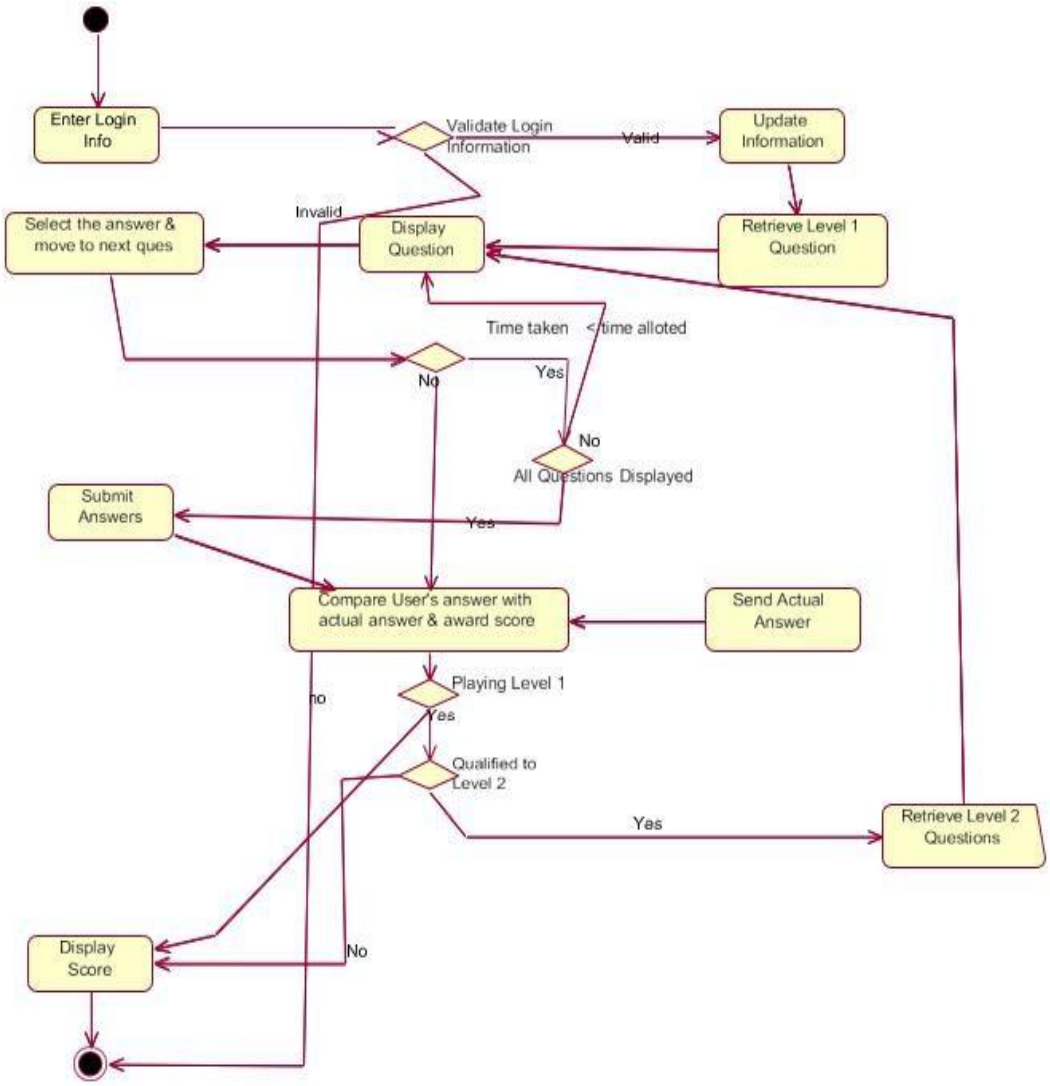
a solid circle. There can only be one Initial State on a diagram.

Final State: Final States mark the end of the modeled workflow. There can be multiple Final States, and these are modeled using a solid circle surrounded by another circle.

Synchronization Bar: Activities often can be done in parallel. To split processing ("fork"), or to resume processing when multiple activities have been completed ("join"), Synchronization Bars are used. These are modeled as solid rectangles, with multiple transitions going in and/or out.

Activity diagram: Quiz System

Participant **Quiz System** **Database**



RESULT: Thus, the relevant state chart diagrams and activity diagrams for Quiz System has been drawn using the ArgoUML.

State Chart Diagram (3 marks)	
Activity Diagram (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

Ex. No: 07 Date:	IMPLEMENTATION OF PARTIAL LAYERED LOGICAL ARCHITECTURE WITH UML PACKAGE DIAGRAM

Aim: To draw the partial layered, logical architecture diagram with UML package diagram notation.

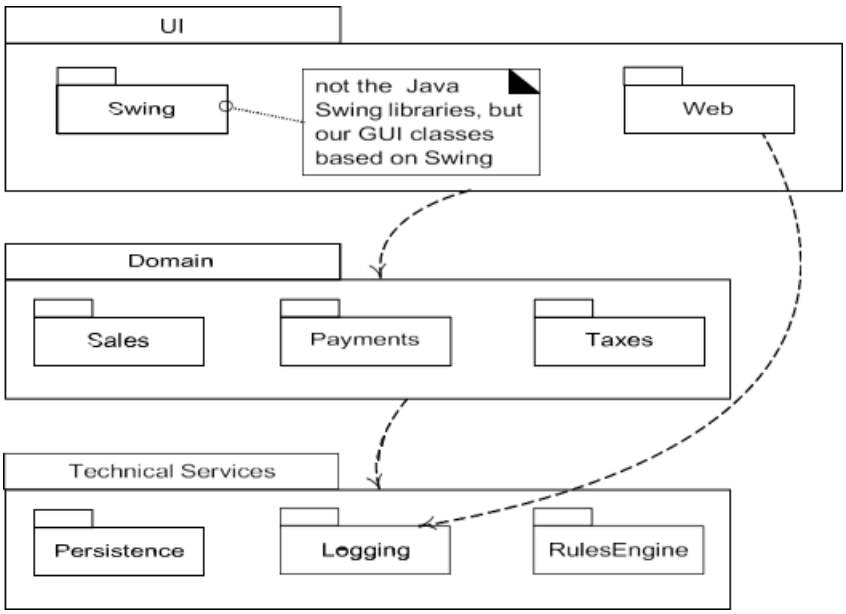
Logical Architecture and Layers

- Logical architecture: the large-scale organization of software classes into packages, subsystems, and layers.
  - "Logical" because no decisions about deployment are implied.
- Layer: a very coarse-grained grouping of classes, packages, or subsystems Typical layers in an OO system:
  - User Interface
  - Application Logic and Domain Objects
  - Technical Services
    - Application-independent, reusable across systems.

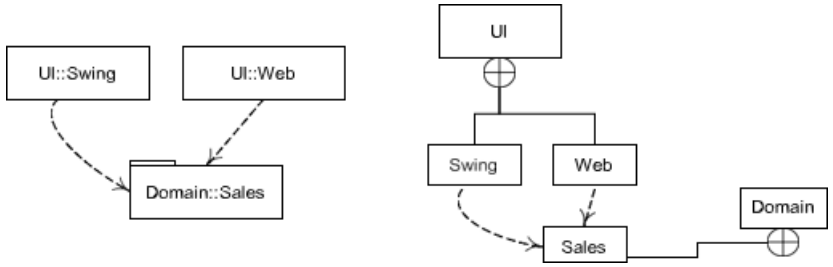
Relationships between layers:

- Strict layered architecture: a layer only calls upon services of the layer directly below it.
- Relaxed layered architecture: a higher layer calls upon several lower layers.

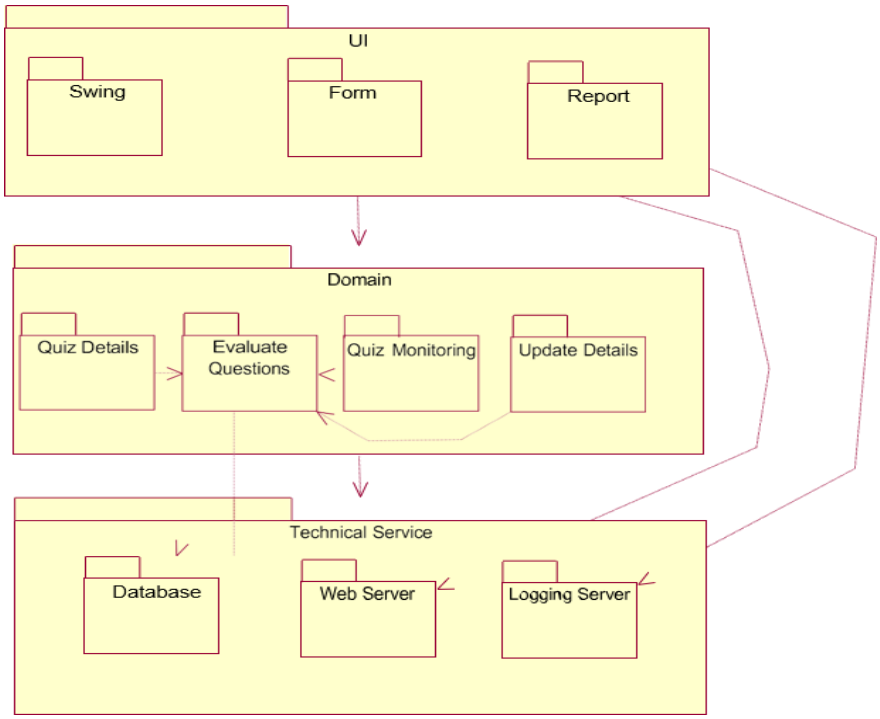
Layers shown with UML package diagram.



Various UML notations for package nesting



Package Diagram: Quiz System



RESULT: Thus, the partial layers and logical architecture are implemented successfully using the UML Package diagram using ArgoUML.

Identifying partial layers and logical architecture (3 marks)	
UML Package Diagram (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

Ex. No: 08 Date:	PROJECT DEVELOPMENT AND TESTING FOR THE QUIZ SYSTEM

Aim: To develop and test the technical services layer, domain objects layer and User interface layer.

Description:

In a quiz system we perform the following

1) Student registration
2) Quiz Participation
3) Result Processing

1) Student Registration

The inputs to the quiz system initially is the student details for the quiz which involves the following details like

1) name
2) year
3) branch
4) college name

When a student enters these details it will be updated in a database. After that the student will be allowed to participate.

2) Quiz Participation

When the participant begins to answer the questions, a clock is maintained to keep track of the time.

The correct answers for the questions are stored in a separate database. After answering the questions in the preliminary round, the system verifies whether the student has answered the minimum number of questions in each level. If not, a message is displayed to make the student answer the required number of questions within an allotted time.

If the student qualifies the prelims, he/she can proceed to the finals in the same way.

3) Result Processing

After all the students have participated in the quiz, the system processes the marks scored by all the students. The system sorts the marks and generates a final output, which displays the name and scores

SOFTWARE DEVELOPMENT AND TESTING

TECHNICAL SERVICES LAYER

Student Registration

Field Name	Data Type
Name	Text
Branch	Text
College	Text

Level 1-Questions and Answers

Field Name	Data Type
qno	Number
question	Text
option1	Text
option2	Text
option3	Text
option4	Text
ans	Text
rightans	Text

Level 2: Questions and answers

Field Name	Data Type
qno	Number
question	Text
option1	Text
option2	Text
option3	Text
option4	Text
ans	Text
rightans	Text

UI LAYER AND DOMAIN LAYER

Testing UI Layer & Domain Layer

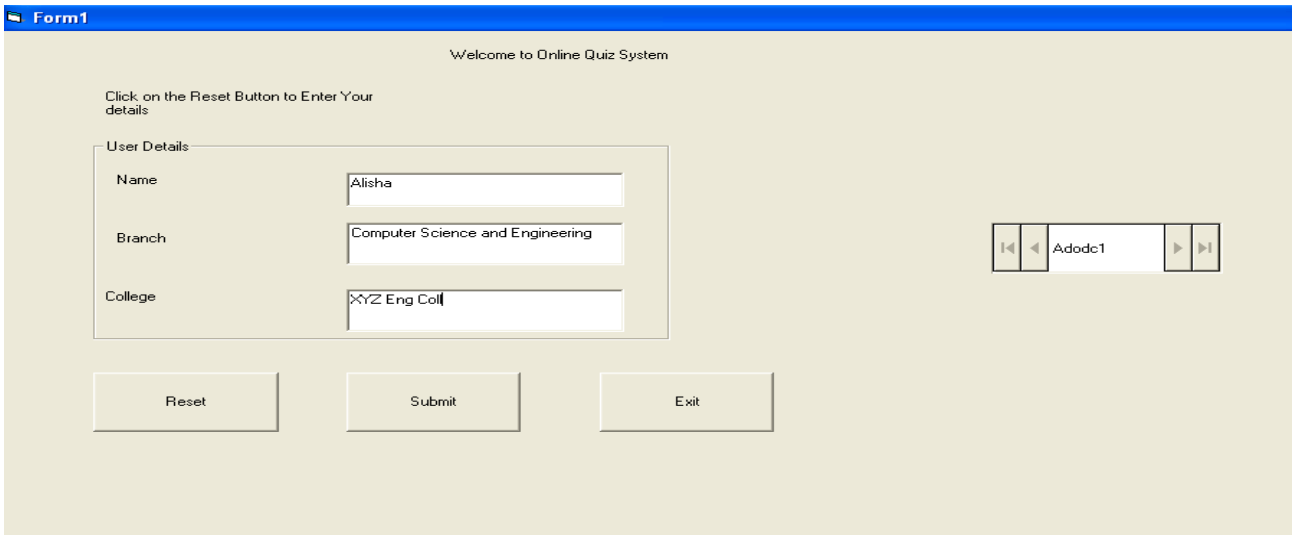
Test case Template

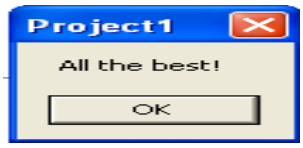
Test case Id : Screen001

Product : Quiz System

Module : Enter Student details

Form 1: Student Registration.





S.No	Action To Perform	Expected Result	Actual Result	Pass/Fail
1	Run the executable file	Enter Student details form appears	As expected	Pass
2	Click reset Button	Resets the text fields	-do-	Pass

3.	Enter “Alisha”, ”CSE”,”XYZ” in Name ,Branch and College Text Box	The Cursor moves to the submit button	-do-	Pass
4.	Click Submit Button	The Student details window is closed and goes to the level 1 of the quiz form	-do-	Pass
5.	Click Cancel Button	The Student details form is closed	-do-	Pass

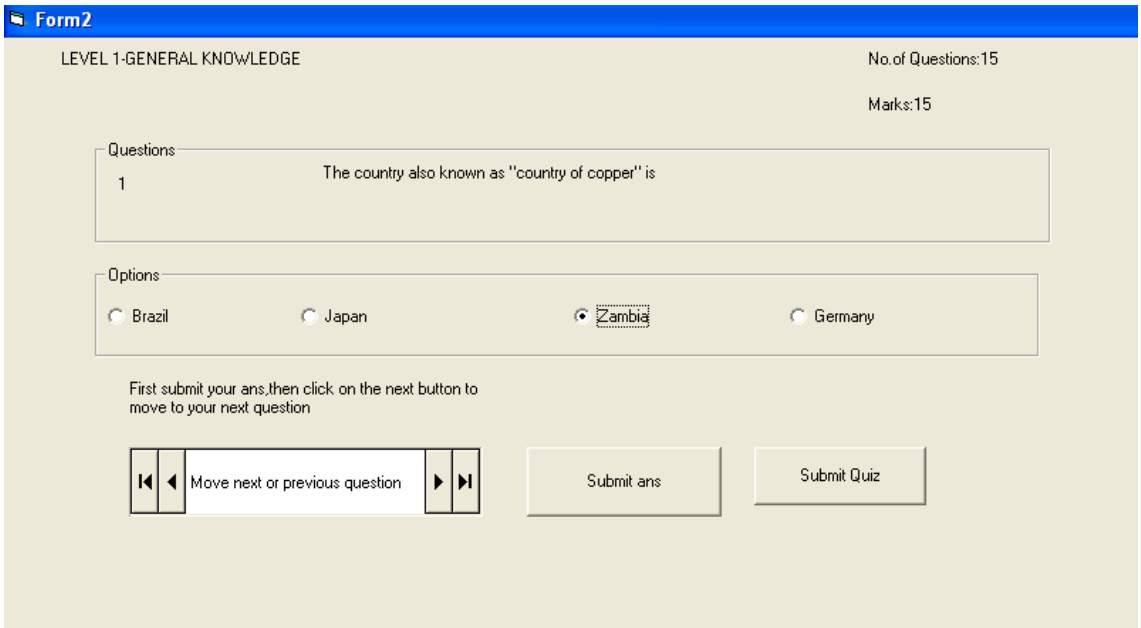
Test case Template

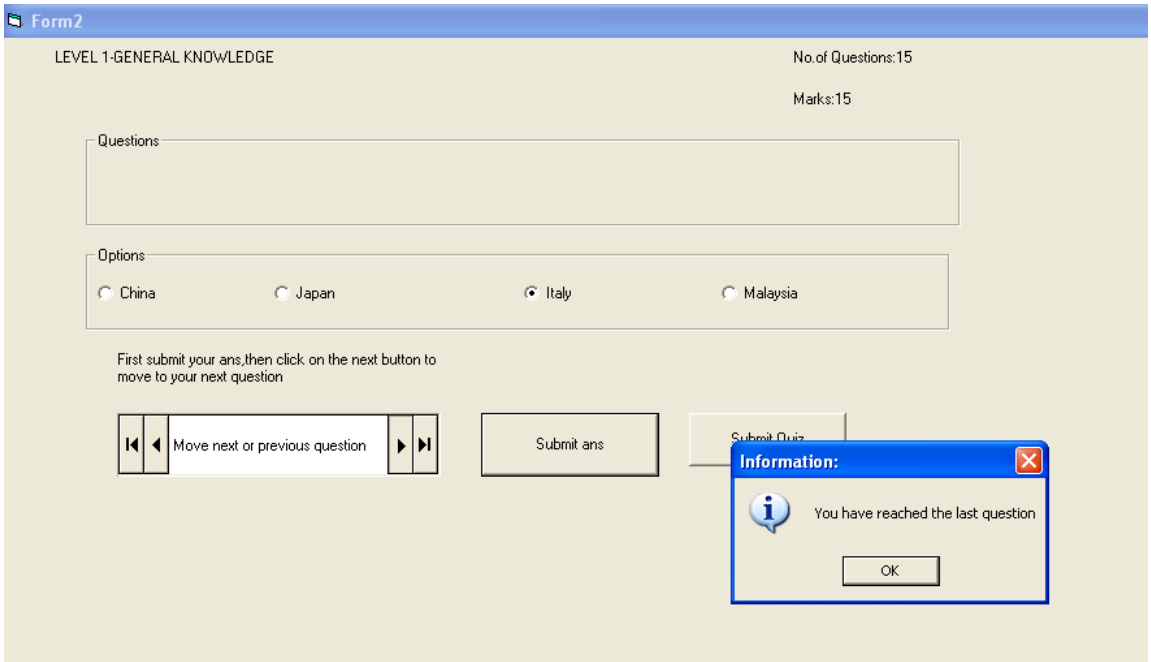
Test case Id : Screen002

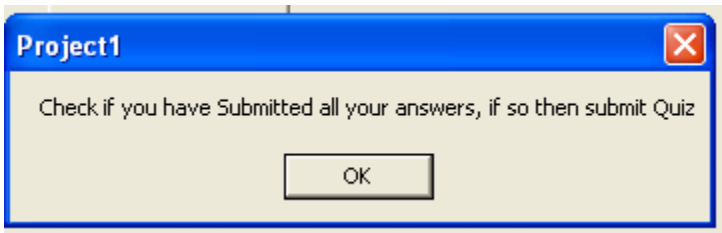
Product : Quiz System

Module : Level 1 of the Quiz

Form 2: Level 1







S.No	Action To Perform	Expected Result	Actual Result	Pass/Fail
1.	Select the suitable option for the required question.	1. The cursor moves to the Submit ans button. 2. The Cursor Moves to the next question using ADODC control	-do-	Pass
2.	Click submit ans Button	The ans is accepted and stored in the database.	-do-	Pass
3.	Click submit quiz Button	All the ans are accepted and the quiz is submitted. The Level 1 of the quiz form is closed.	-do-	Pass

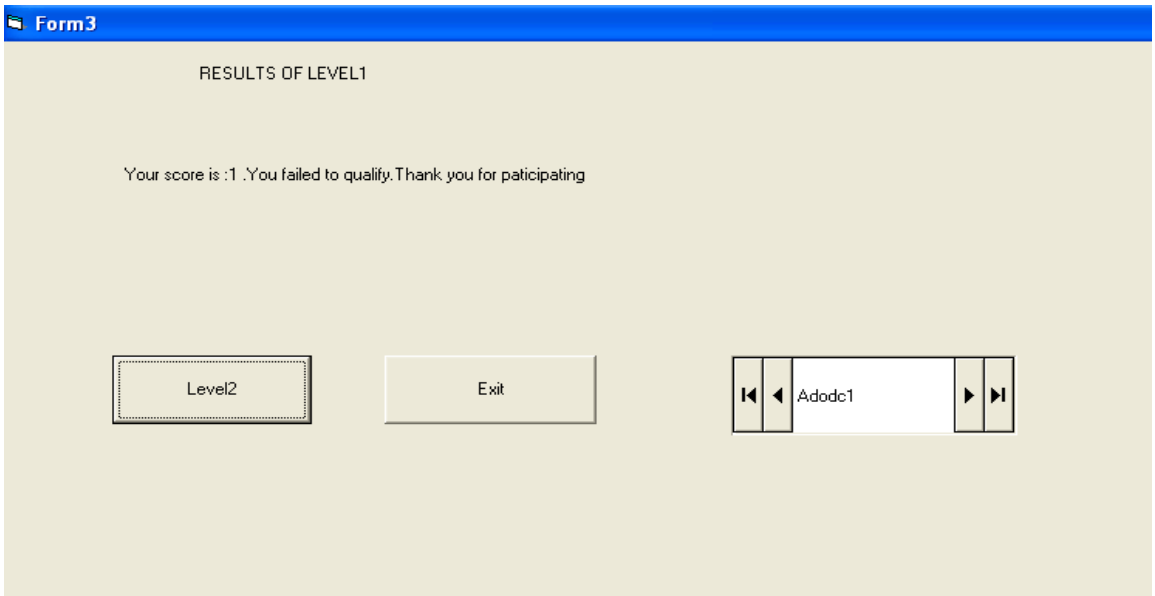
Test case Template

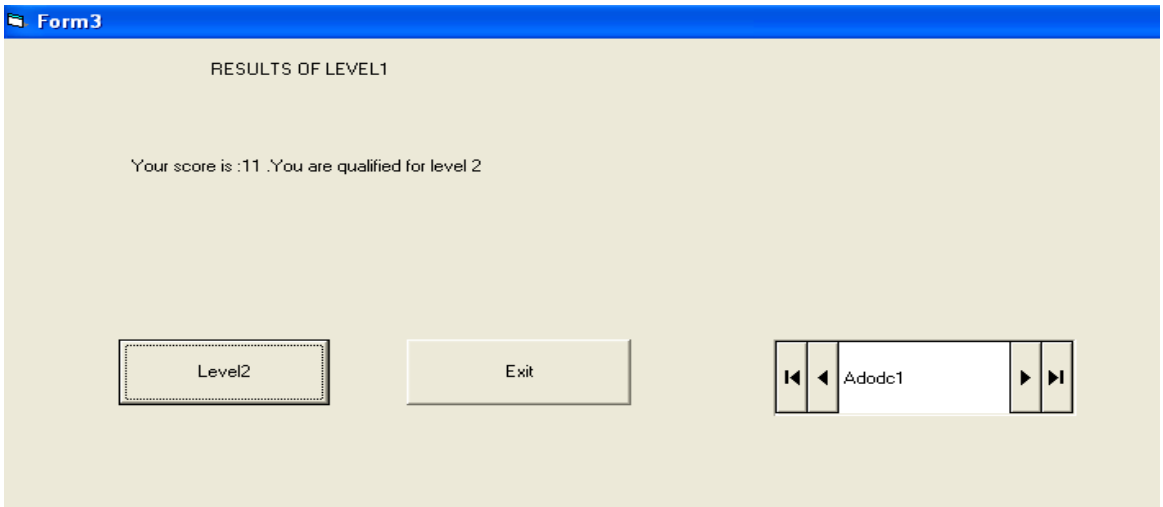
Test case Id : Screen003

Product : Quiz System

Module : Result Processing of Level 1 of the Quiz

orm 3: Result Processing of Level 1





S.No	Action To Perform	Expected Result	Actual Result	Pass/Fail
1.	Click the Level 2 Button	1. Error Message appears as "You can Exit, Click on the Exit Button if not qualified. 2. If qualified close result form and enter Level 2 Form of the quiz.	-do-	Pass
2.	Click Exit Button	The Result Processing Form is closed and the Level 2 Form appears	-do-	Pass

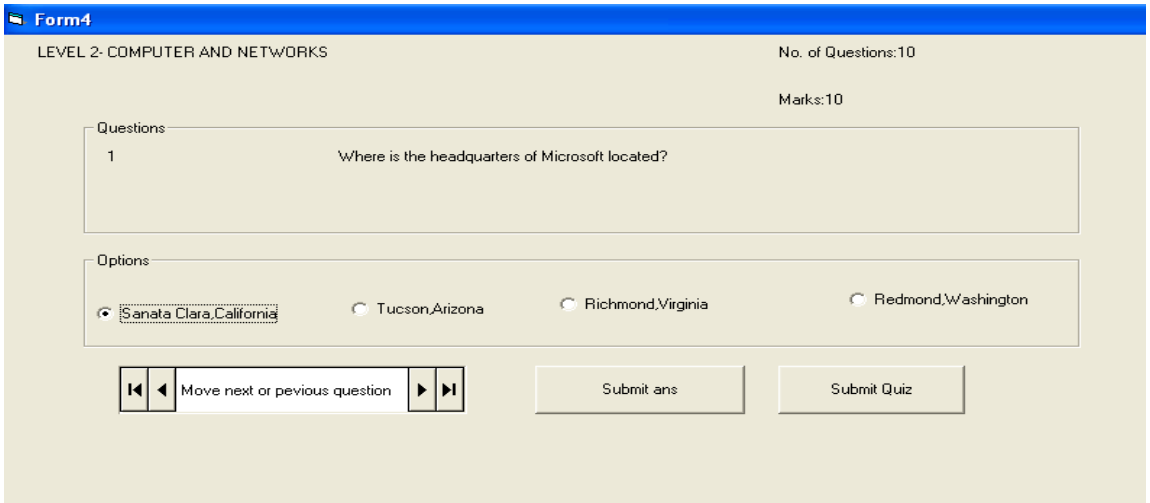
Test case Template

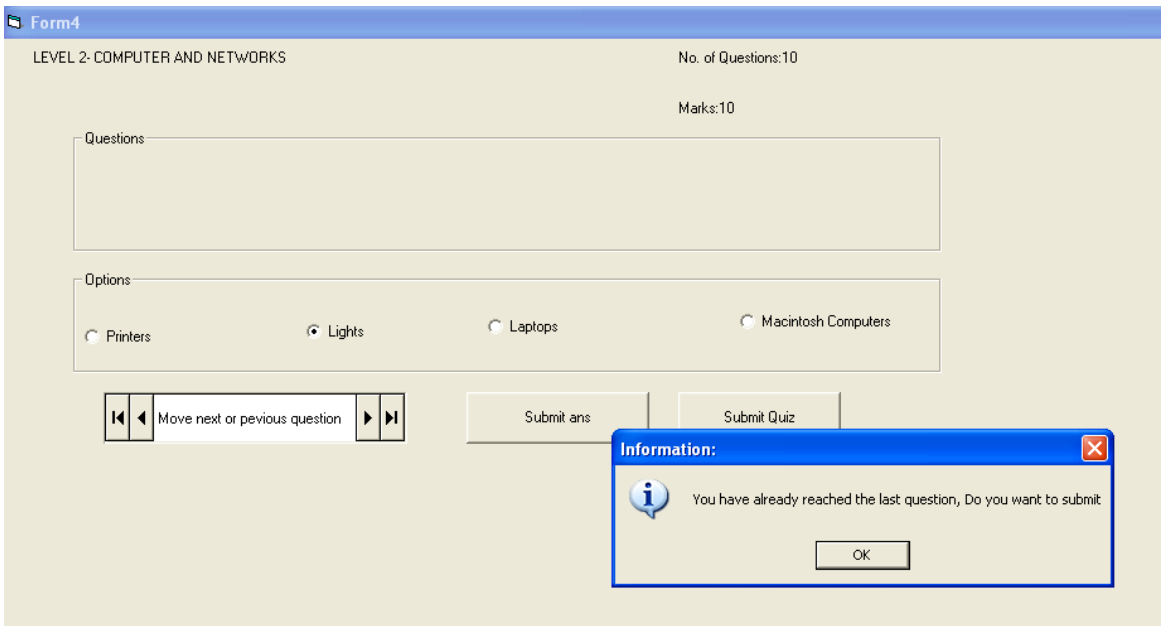
Test case Id : Screen004

Product : Quiz System

Module : Level 2 of the Quiz

Form 4: Level 2 of the quiz

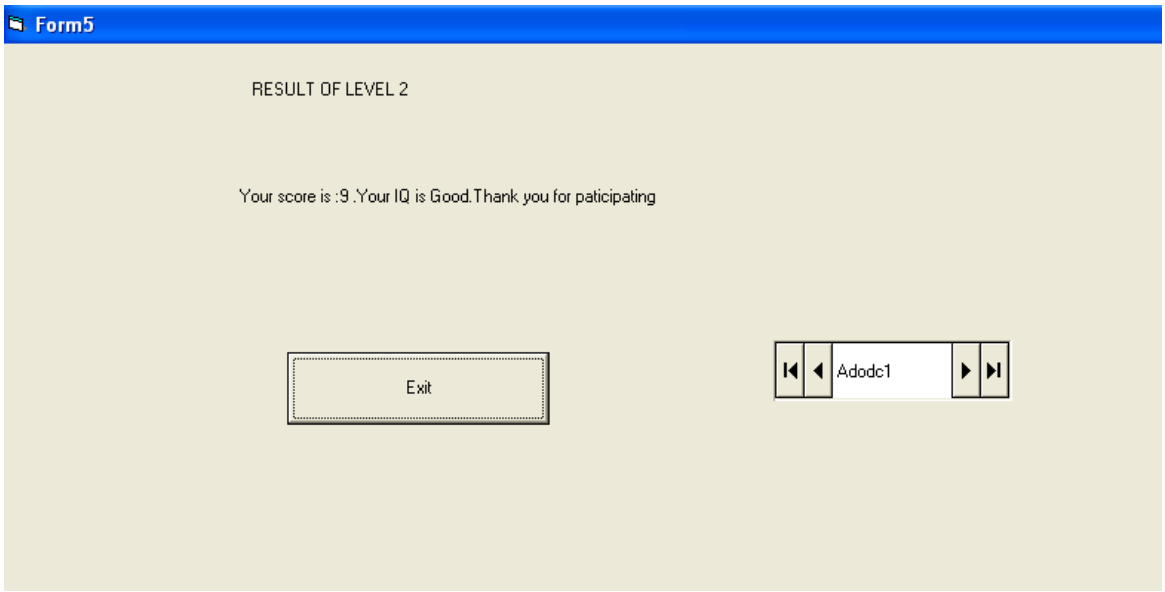




S.No	Action To Perform	Expected Result	Actual Result	Pass/Fail
1.	Select the suitable option for the required question.	1. The cursor moves to the Submit ans Button. 2. The cursor moves to the next question using ADODC control	-do-	Pass
2.	Click submit ans Button	The ans is accepted and stored in the database.	-do-	Pass
3.	Click submit quiz Button	The ans are accepted and the quiz is submitted The Level 2 of the quiz form is closed.	-do-	Pass

Test case Template
Test case Id : Screen005
Product : Quiz System
Module : Result Processing of Level 2 of the Quiz

Form 5: Result Processing of Level 2



S.No	Action To Perform	Expected Result	Actual Result	Pass/Fail
1.	Click Exit Button	The Result Processing Form is closed and a Message Window Appears as “You can exit”	-do-	Pass

RESULT: Thus, the software system for all the scenarios is identified as per the use case diagram has been successfully tested.

Identify the use-case diagram (3 marks)	
Testing the software system for all the scenarios (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

Ex. No: 09 Date:	IMPROVE THE REUSABILITY AND MAINTAINABILITY OF THE SOFTWARE SYSTEM BY APPLYING APPROPRIATE DESIGN PATTERNS

AIM:

To Improve the reusability and maintainability of the software system by applying appropriate design patterns.

Creational Patterns:

These help in object creation in a manner suitable to the situation.

- **Factory Method Pattern**
  - *Use when*: You want to delegate object creation to subclasses.
  - *Benefit*: Promotes loose coupling by reducing the dependency on concrete classes.
  - *Example*: In a GUI system, use a factory to create different buttons for Windows and macOS.
- **Singleton Pattern**
  - *Use when*: A class must have only one instance (e.g., configuration manager, logging service).
  - *Benefit*: Ensures controlled access and global availability.
- **Builder Pattern**
  - *Use when*: Creating complex objects with many optional fields.
  - *Benefit*: Simplifies object construction and improves readability.

Structural Patterns:

These deal with the composition of classes or objects.

- **Adapter Pattern**
  - *Use when*: You want to use an existing class but its interface is incompatible.
  - *Benefit*: Enhances reusability by adapting different interfaces.
- **Facade Pattern**
  - *Use when*: You want to provide a simplified interface to a complex subsystem.
  - *Benefit*: Improves maintainability by isolating clients from complex logic.
- **Decorator Pattern**
  - *Use when*: You want to add responsibilities to individual objects dynamically.
  - *Benefit*: Promotes open/closed principle — open for extension but closed for modification.

Behavioral Patterns:

These are concerned with communication between objects.

- **Observer Pattern**
  - *Use when*: Multiple objects need to be informed of state changes in another object.
  - *Benefit*: Supports event-driven programming and loose coupling.
- **Strategy Pattern**
  - *Use when*: You have multiple algorithms for a task and want to switch them at runtime.
  - *Benefit*: Increases flexibility and reduces duplicated code.
- **Command Pattern**
  - *Use when*: You need to parameterize objects with operations or queue requests.

- *Benefit*: Decouples sender from receiver and enables undo/redo functionality.

General Tips for Maintainable Code Using Patterns:

- **Use interfaces and abstractions** rather than concrete classes.
- **Follow SOLID principles**, which many design patterns reinforce.
- **Document design decisions** so future maintainers understand the rationale.
- **Modularize code** using patterns like Facade and Strategy for better testability and readability.

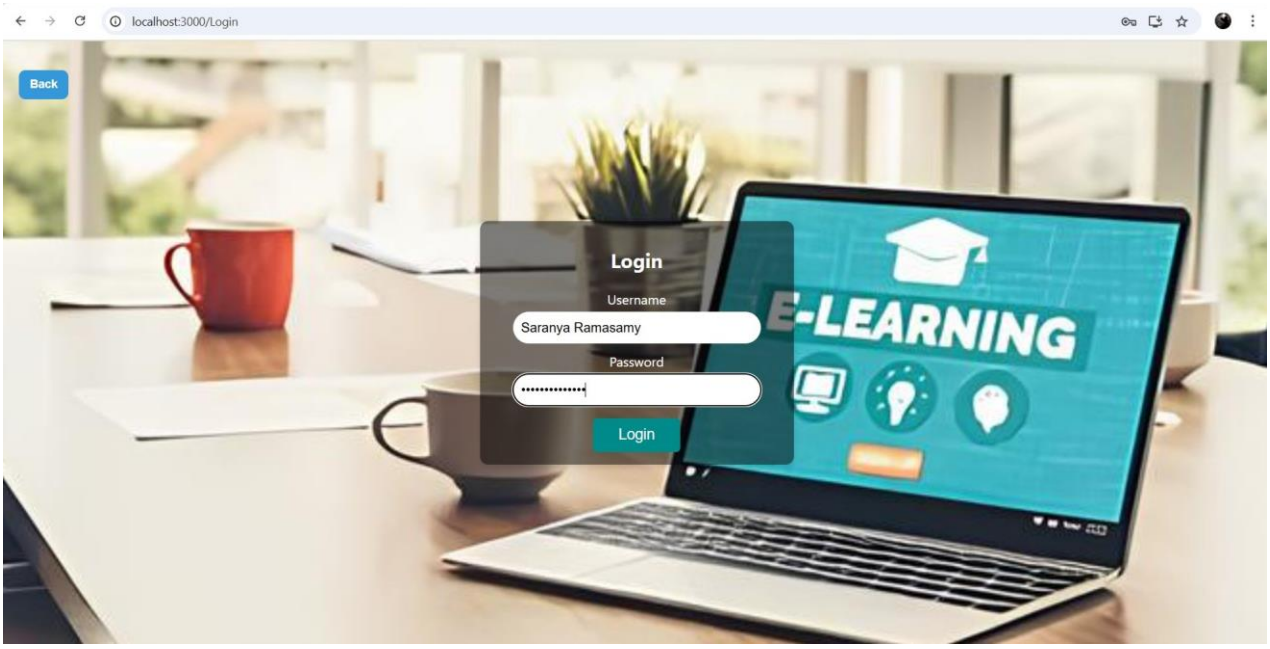
RESULT: Thus, the steps are taken to improve the reusability and maintainability of the software system by applying appropriate design patterns.

Identified steps to improve the reusability and maintainability (3 marks)	
Applying appropriate design patterns (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	

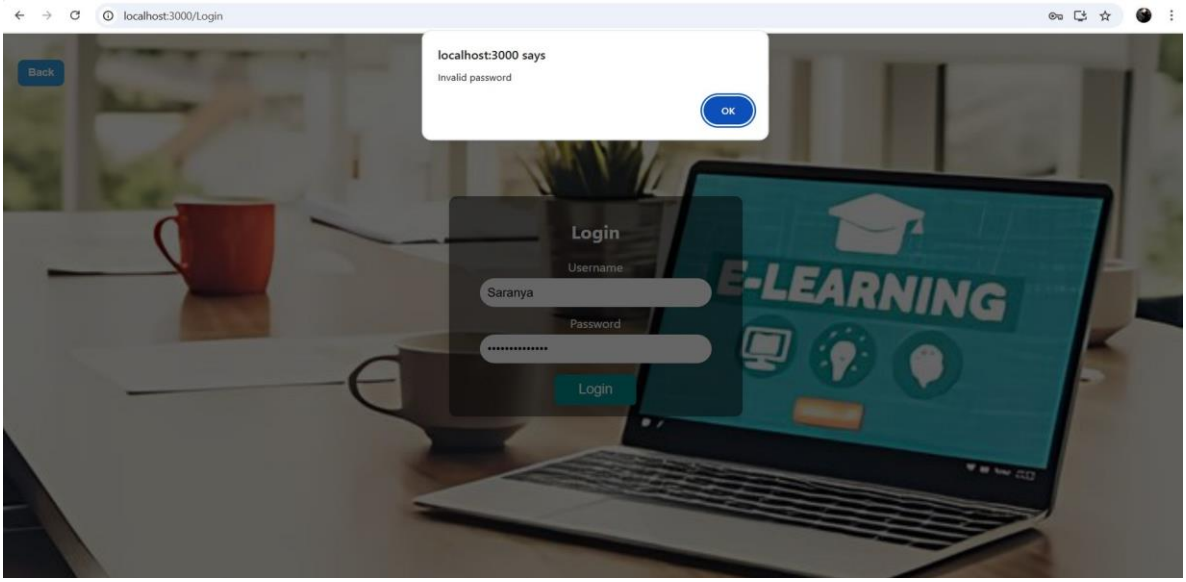
Ex. No: 10 Date:	IMPLEMENT THE MODIFIED SYSTEM AND TEST IT FOR VARIOUS SCENARIOS

Aim: To implement the modified system and test it for various scenarios.

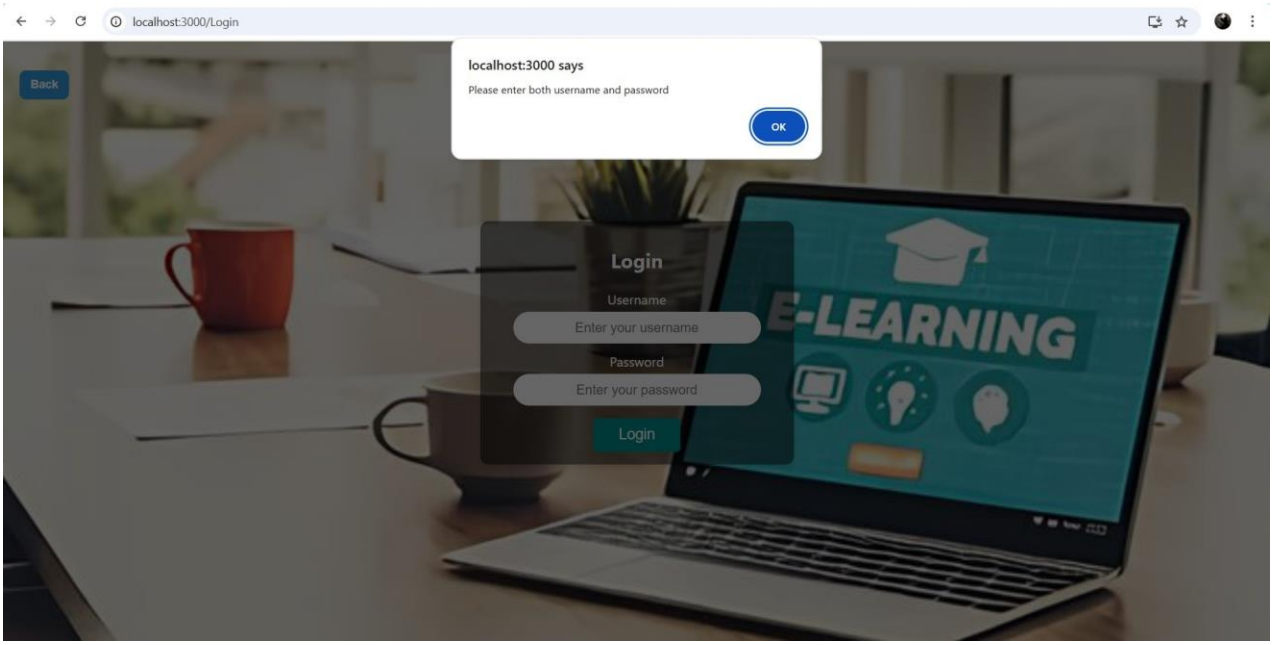
Output:



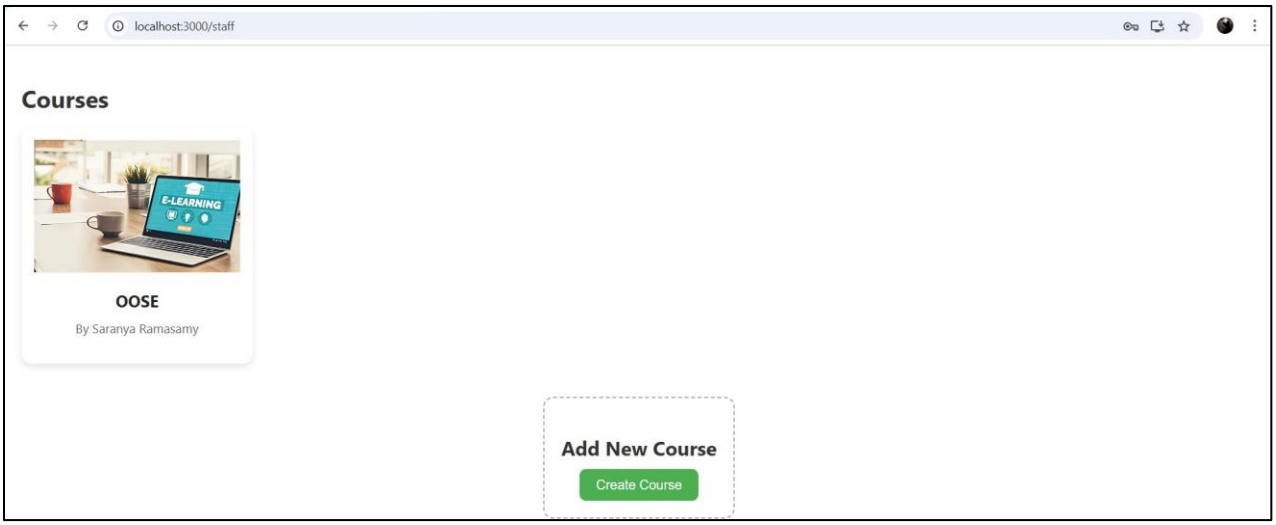
Before Manual testing User Register Form



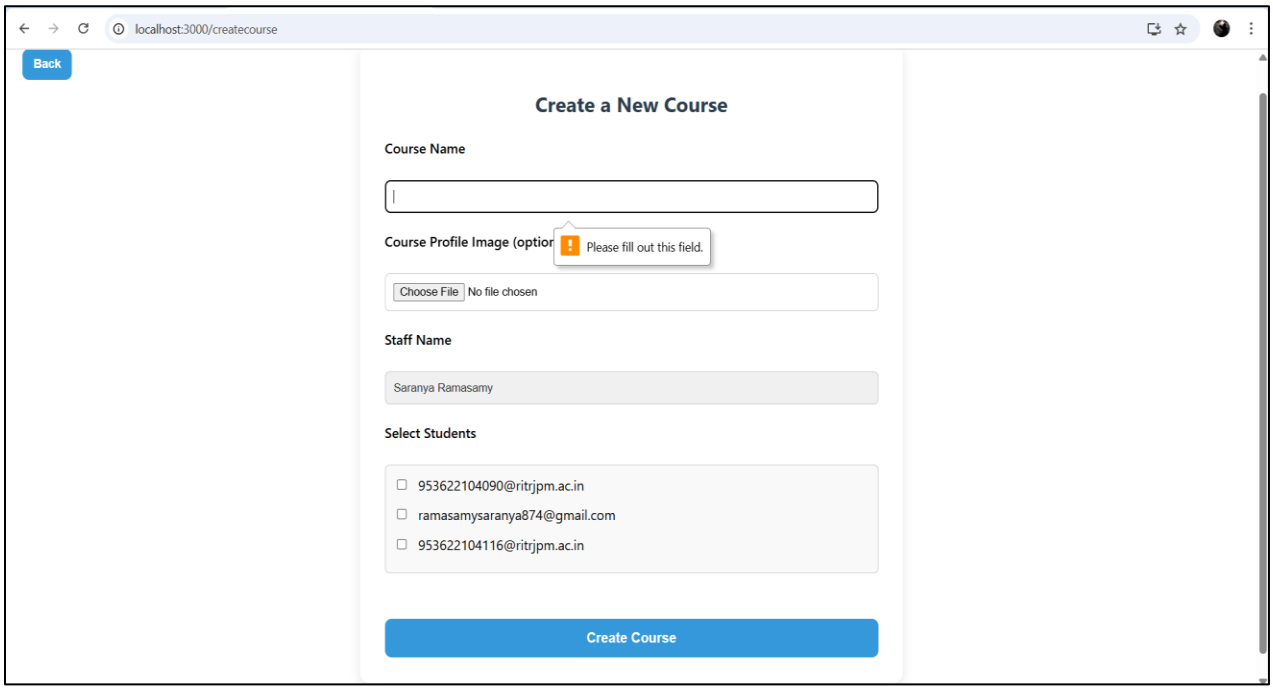
Password validation testing



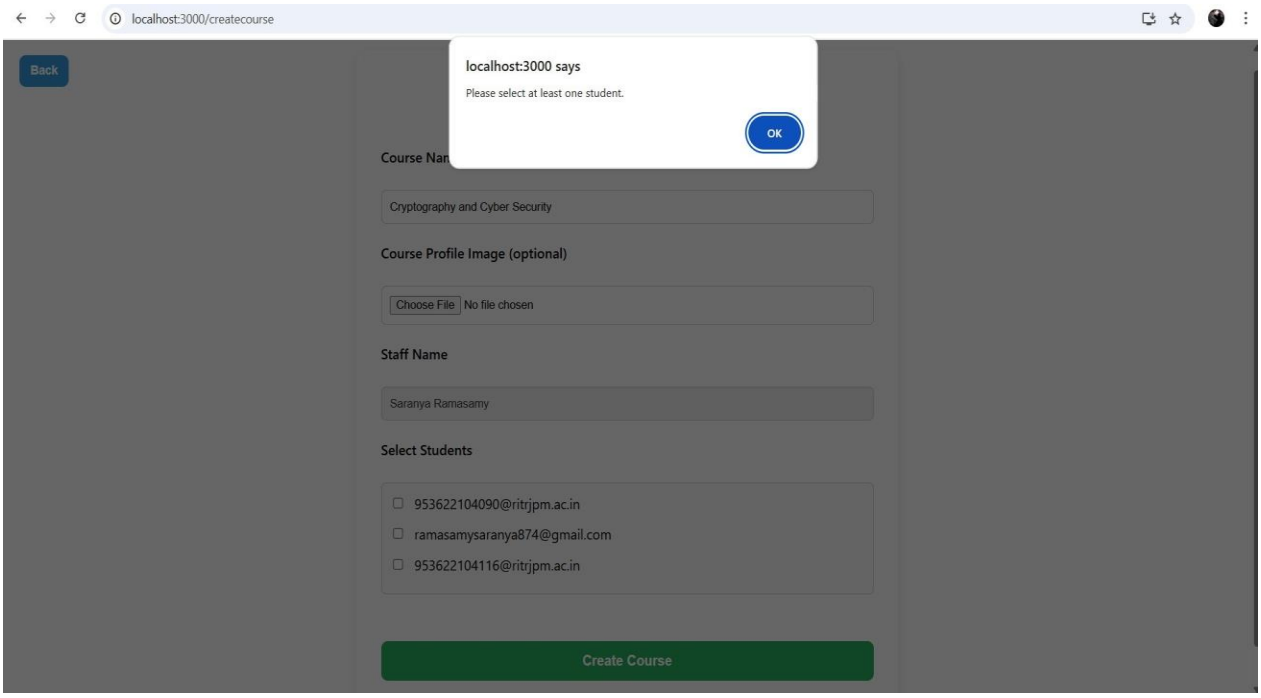
Null Submission testing



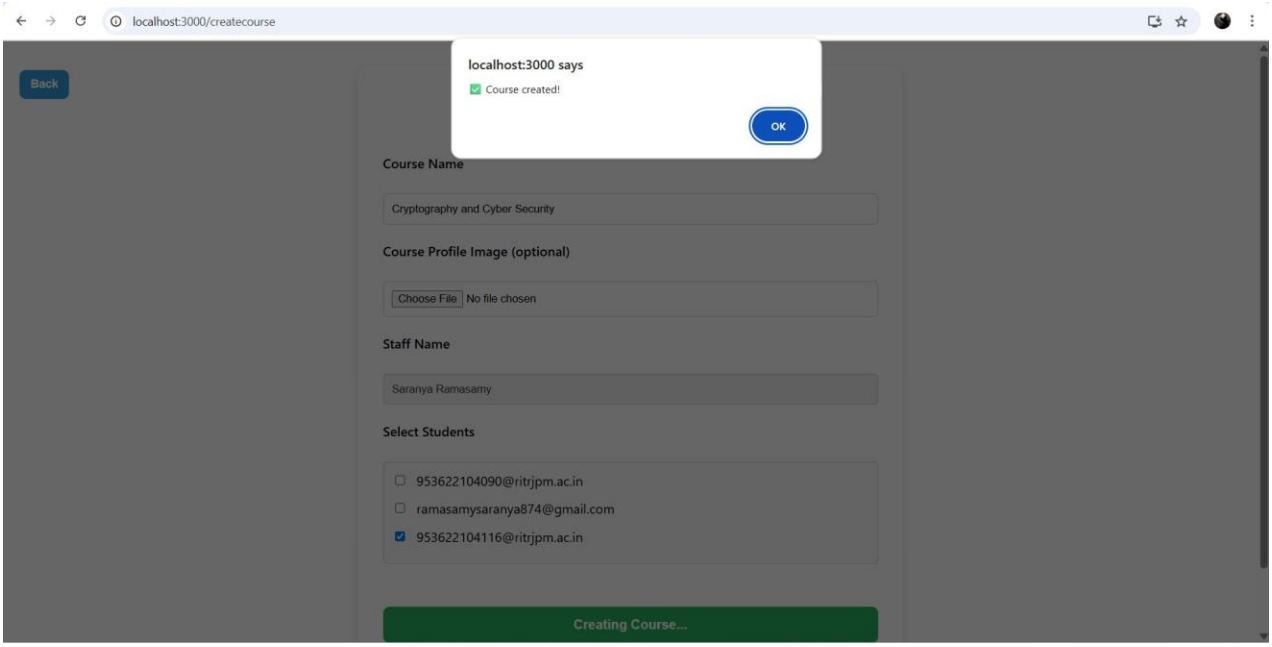
Successfully Logged after entering valid credentials



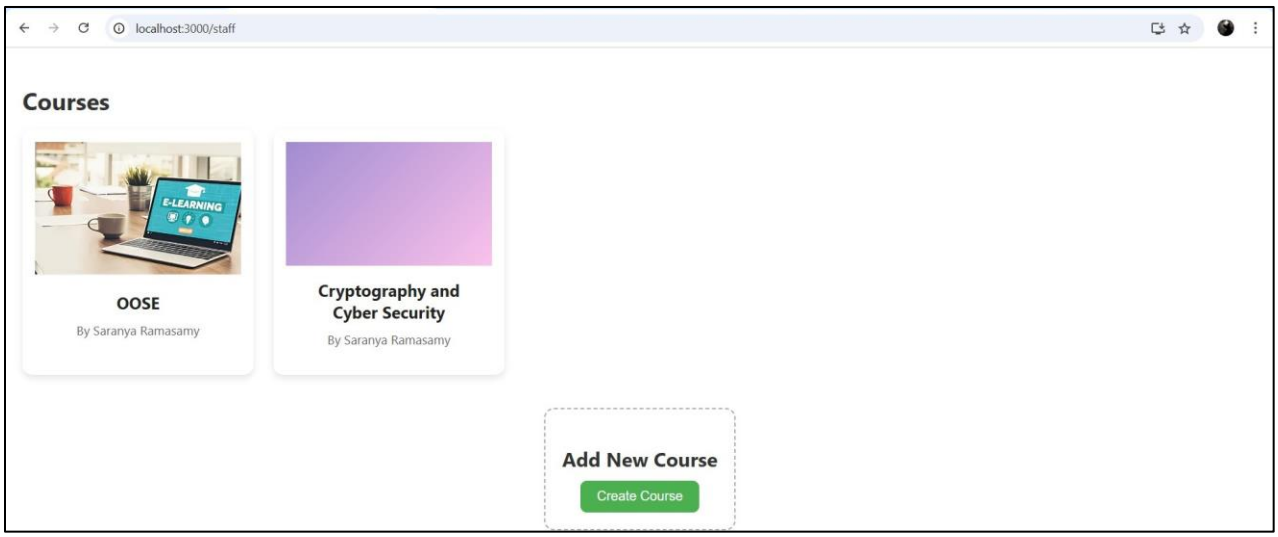
Validate that the staff while creating a course.



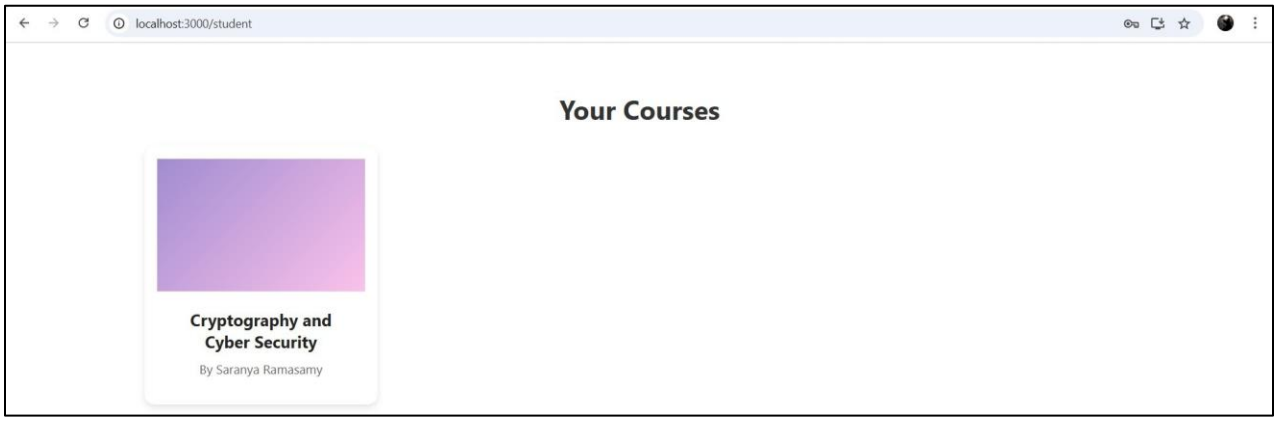
Adding student in the course is tested



while the staff successfully created the course



After the course create it added successfully in the staff page.



After the course added in the staff it also shown in the student login also

RESULT: Thus, the modified system has been successfully implemented by testing it on various scenarios.

Implementation of modified system (3 marks)	
Testing various scenarios (3 marks)	
Output & Viva Explanation (4 marks)	
Total (10 marks)	